

HACKATHON — DevWeek 26
Ingeniería de Software — Nivel Profesional

FoodLoop

Red Social Anti-Desperdicio de Alimentos

Documento de Análisis, Diseño y Especificación del Sistema

Proyecto FoodLoop PWA	Reto RETO 2 — SmartFood
Modalidad Colaborativa por equipos	Horario 2:00 p.m. — 8:30 p.m.
Tecnologías Next.js · Supabase · Gemini Flash API	Fecha Mayo 2026

1. Descripción del Problema

1.1 Contexto General

El desperdicio de alimentos representa uno de los problemas más urgentes de la actualidad a nivel económico, social y ambiental. En México, se estima que alrededor del 35% de los alimentos producidos se desperdician antes de llegar a ser consumidos. En el contexto universitario, esta problemática se agudiza: los estudiantes foráneos que viven solos o en pequeños departamentos compran alimentos sin planeación, no saben cómo conservarlos adecuadamente y terminan descartándolos antes de que caduquen.

Las cafeterías cercanas al campus enfrentan una situación similar: al cierre del día tienen excedentes de comida preparada que deben desechar por normativa, a pesar de que aún son aptos para el consumo. Esta situación representa una pérdida económica directa para los establecimientos y contribuye innecesariamente a la generación de residuos orgánicos.

1.2 Causas Principales

- Compras excesivas o impulsivas sin planeación semanal
- Falta de control de inventario de alimentos en el hogar
- Desconocimiento de técnicas de conservación adecuadas para cada alimento
- Ausencia de un canal eficiente para redistribuir excedentes de comida antes de que expiren
- Cafeterías sin mecanismo para compartir sobrantes del día con la comunidad

1.3 Impacto Negativo del Problema

Dimensión	Impacto
Económico	Perdida de dinero en alimentos no consumidos para estudiantes y comerciantes
Ambiental	Mayor generación de residuos orgánicos, emisión de metano en rellenos sanitarios
Social	Estudiantes con presupuesto limitado no tienen acceso a alimentos que otros desechan
Educativo	Ausencia de herramientas que promuevan hábitos responsables de consumo

2. Descripción de la Solución

2.1 Visión del Producto

FoodLoop es una aplicación web progresiva (PWA) con dinámica de red social diseñada para reducir el desperdicio de alimentos en comunidades universitarias. Permite a estudiantes foráneos, residentes cercanos y cafeterías publicar alimentos disponibles antes de que caduquen, y a otros usuarios reclamarlos de forma gratuita o a bajo costo.

Analogía	FoodLoop funciona como un 'Facebook de la comida': los usuarios publican lo que tienen de sobra (como si fuera una publicación en un muro), otros lo ven en su feed cronológico, y pueden 'reclamarlo' antes de que expire. La IA actúa como un chef personal que sugiere recetas con lo que tienes y te enseña cómo conservar mejor tus alimentos.
-----------------	---

2.2 Propuesta de Valor

- Para el donante: reduce el desperdicio
- Para el receptor: accede a alimentos gratuitos o a muy bajo costo
- Para las cafeterías: un canal para poder publicar sus productos estancados o próximos a expirar
- Para la comunidad: fomenta la cultura de aprovechamiento y economía circular

2.3 Funcionalidades Principales

- Feed tipo red social con publicaciones de alimentos disponibles
- Creación de publicaciones con foto, descripción, fecha de caducidad y punto de entrega
- Sistema de reclamo y coordinación de entrega entre usuarios
- Asistente de IA para generación de recetas con ingredientes disponibles
- Asistente de IA para consejos de almacenamiento y conservación
- Gestión del estado de publicaciones (Disponible / Reclamado / Entregado / Expirado)
- Sistema de notificaciones para coordinar entregas
- Instalable como PWA en dispositivos móviles y de escritorio

3. Objetivos del Proyecto

3.1 Objetivo General

Desarrollar una aplicación web progresiva (PWA) que funcione como red social de redistribución de alimentos, permitiendo a estudiantes universitarios, residentes y cafeterías publicar y reclamar excedentes de comida antes de su fecha de caducidad, apoyada por un asistente de inteligencia artificial que reduzca el desperdicio mediante recetas y consejos de conservación.

3.2 Objetivos Específicos

- Implementar un sistema de autenticación seguro con soporte para correo electrónico
- Crear un feed dinámico de publicaciones de alimentos ordenado por proximidad de caducidad o fecha de publicación
- Desarrollar el módulo de publicación con carga de imagen, descripción, fecha de caducidad y punto de entrega
- Implementar el flujo de reclamo de alimentos con notificaciones entre donante y receptor
- Integrar la API de Gemini Flash para generación de recetas con ingredientes ingresados por el usuario
- Integrar la API de Gemini Flash para asesoramiento de almacenamiento y conservación de alimentos
- Configurar la aplicación como PWA instalable con service worker y manifest válido
- Desplegar la aplicación en la nube con disponibilidad continua (24/7)

4. Usuarios Objetivo

El sistema contempla tres tipos de usuario con diferentes roles, necesidades y permisos dentro de la plataforma:

4.1 Estudiante Foraneo / Residente

4.2 Cafeteria / Negocio Local

4.3 Administrador del Sistema

5. Requerimientos Funcionales

Los requerimientos funcionales describen las capacidades específicas que el sistema debe poseer para satisfacer las necesidades de los usuarios. Se identificaron 10 requerimientos funcionales con el siguiente formato estándar:

RF01 — Autenticación de Usuarios

Descripción	El sistema debe permitir el registro e inicio de sesión de usuarios mediante correo electrónico con contraseña, o mediante autenticación OAuth (Google). El registro solicitará: nombre, correo, tipo de usuario (Estudiante / Cafetería) y contraseña.
Actor	Estudiante, Cafetería
Entrada	Correo electrónico, contraseña / token OAuth de Google
Salida	Sesión iniciada, redirección al feed principal, token JWT de sesión
Prioridad	Alta

RF02 — Publicación de Alimentos

Descripción	El sistema debe permitir a un usuario autenticado crear una publicación ofreciendo un alimento. La publicación incluirá: fotografía (obligatoria), título, descripción, fecha de caducidad o consumo preferente, punto de entrega (texto libre o coordenadas), tipo de oferta (gratuito / precio simbólico) y etiquetas de categoría.
Actor	Estudiante, Cafetería
Entrada	Imagen, título, descripción, fecha, punto de entrega, precio, categorías
Salida	Publicación creada y visible en el feed con estado 'Disponible'
Prioridad	Alta

RF03 — Feed de Publicaciones

Descripción	El sistema debe mostrar un feed cronológico con las publicaciones de alimentos disponibles. El feed ordenará por defecto por proximidad de caducidad (los que caducan antes aparecen primero) y permitirá filtrar por categoría o tipo de publicante. Sólo se mostrarán publicaciones con estado 'Disponible'.
Actor	Estudiante, Cafetería
Entrada	Solicitud de carga del feed, filtros opcionales
Salida	Lista de tarjetas de publicaciones con imagen, título, tiempo restante y publicante

Prioridad	Alta
-----------	------

RF04 — Reclamo de Alimentos

Descripción	El sistema debe permitir a un usuario autenticado 'reclamar' una publicación disponible. Al reclamar, la publicación cambia a estado 'Reclamado', se notifica al publicante y se habilita un mini-chat de coordinación entre ambas partes para acordar la entrega. Solo un usuario puede reclamar por publicación.
Actor	Estudiante (como receptor)
Entrada	Acción de reclamar sobre una publicación 'Disponible'
Salida	Estado de publicación actualizado a 'Reclamado', notificación al publicante, chat habilitado
Prioridad	Alta

RF05 — Asistente IA de Recetas

Descripción	El sistema debe integrar un módulo de asistente conversacional con IA (Gemini Flash) donde el usuario ingrese una lista de ingredientes o sobras disponibles y reciba una receta detallada paso a paso para prepararlos, incluyendo tiempos estimados y porciones.
Actor	Estudiante, Cafetería
Entrada	Lista de ingredientes en texto libre (ej. 'tengo arroz, zanahoria y pollo')
Salida	Receta generada con nombre del platillo, ingredientes con cantidades, pasos, tiempo y porciones
Prioridad	Alta

RF06 — Asistente IA de Almacenamiento

Descripción	El sistema debe permitir al usuario consultar al asistente de IA (Gemini Flash) sobre cómo guardar correctamente un alimento específico para prolongar su vida útil. La respuesta incluirá método de almacenamiento, temperatura recomendada, contenedor sugerido y tiempo de duración aproximado.
Actor	Estudiante, Cafetería
Entrada	Nombre o descripción del alimento en texto libre (ej. '?Como guardo medio aguacate?')
Salida	Instrucciones claras de conservación con duración estimada y consejos prácticos
Prioridad	Media-Alta

RF07 — Gestión del Estado de Publicaciones

Descripción	El usuario creador de una publicación debe poder cambiar manualmente el estado de su publicación. Los estados válidos son: Disponible (inicial), Reclamado (automático al ser reclamado), Entregado (manual, confirma que la entrega se realizó) y Expirado (manual o automático al pasar la fecha de caducidad). Las publicaciones en estado 'Entregado' o 'Expirado' no aparecen en el feed principal.
Actor	Estudiante, Cafetería (como publicante)
Entrada	Selección del nuevo estado desde el panel de mis publicaciones
Salida	Estado actualizado, publicación removida del feed si aplica
Prioridad	Alta

RF08 — Notificaciones en Tiempo Real

Descripción	El sistema debe enviar notificaciones al usuario cuando: alguien reclame su publicación, el reclamante o el publicante envíe un mensaje en el chat de coordinación, y cuando una publicación propia esté próxima a expirar (24 horas antes). Las notificaciones se muestran dentro de la app mediante badge en el icono de campana.
Actor	Estudiante, Cafetería
Entrada	Evento del sistema: reclamo, mensaje de chat, proximidad de expiración
Salida	Notificación visible en la interfaz, badge numérico en icono de campana
Prioridad	Media

RF09 — Perfil de Usuario

Descripción	Cada usuario tendrá un perfil con: nombre, foto de perfil, tipo de cuenta (Estudiante / Cafetería), contador de publicaciones realizadas, contador de alimentos salvados (entregados) y publicaciones activas. El perfil propio será editable.
Actor	Estudiante, Cafetería
Entrada	Navegación al perfil, datos editados en formulario
Salida	Vista de perfil actualizada, datos guardados en base de datos
Prioridad	Media

RF10 — Historial de Actividad

Descripción	El sistema debe mostrar al usuario un historial de sus publicaciones pasadas (publicadas y reclamadas) con sus estados finales,
-------------	---

	permitiendo revisar cuántos alimentos ha donado, cuantos ha recibido y las interacciones de chat correspondientes.
Actor	Estudiante, Cafetería
Entrada	Navegación a la sección de historial desde el perfil
Salida	Lista paginada de actividad pasada con estados, fechas e interlocutores
Prioridad	Baja

6. Requerimientos No Funcionales

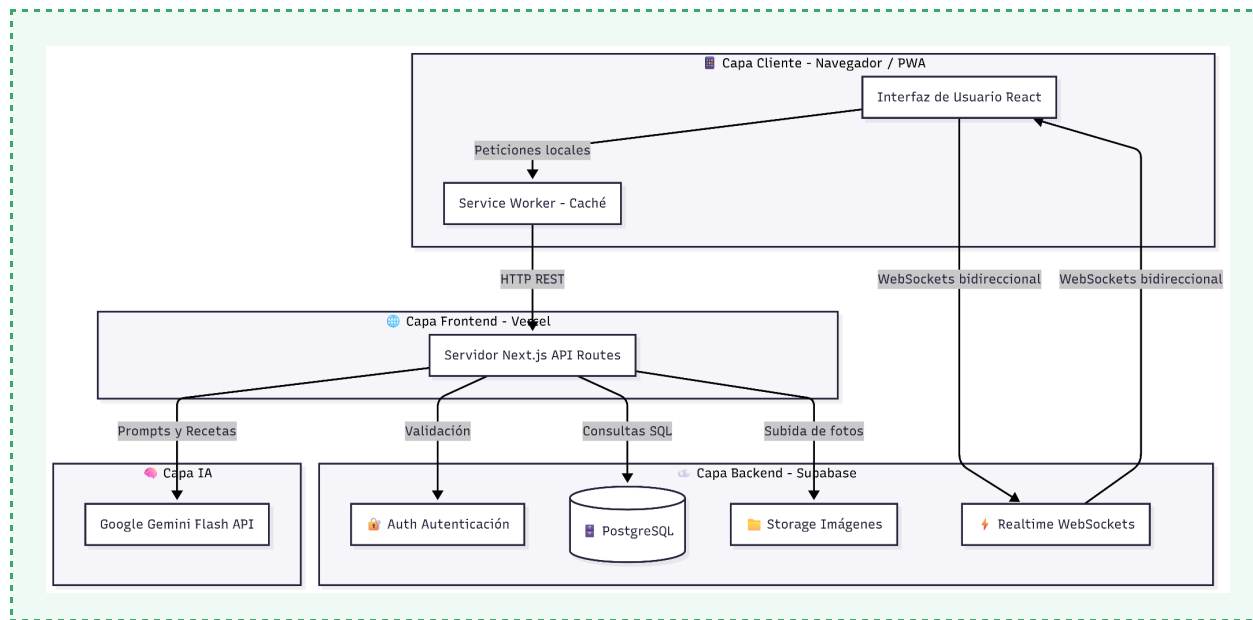
Los requerimientos no funcionales establecen los atributos de calidad del sistema: rendimiento, seguridad, disponibilidad, escalabilidad y usabilidad.

ID	Atributo	Descripcion
RNF01	Usabilidad	La interfaz debe estar diseñada mobile-first con diseño responsive. Las acciones principales (publicar, reclamar, consultar IA) deben completarse en 3 pasos o menos. Cumplir principios de accesibilidad WCAG 2.1 nivel A.
RNF02	Rendimiento IA	El tiempo de respuesta del asistente de IA (recetas y almacenamiento) no debe exceder 5 segundos bajo condiciones normales de red (conexión 4G o superior). El feedback de carga debe mostrarse de forma inmediata.
RNF03	Seguridad	Las contraseñas deben almacenarse con hash bcrypt. Los datos de contacto (teléfono, ubicación exacta) solo se compartirán entre las partes involucradas en un reclamo activo. Las rutas privadas requieren una sesión válida. Comunicación HTTPS obligatoria.
RNF04	Disponibilidad	El sistema debe tener una disponibilidad del 99% mensual. El servicio se aloja en infraestructura cloud (Vercel + Supabase) con redundancia automática. Las actualizaciones se realizan con zero-downtime deployment.
RNF05	Escalabilidad	La arquitectura debe soportar hasta 500 usuarios concurrentes sin degradación del servicio durante el MVP. Supabase con plan gratuito soporta hasta 500 conexiones simultáneas. La API de IA implementará rate limiting de 10 solicitudes por usuario por minuto.
RNF06	Instalabilidad	La aplicación debe cumplir los criterios de PWA de Google: manifest.json válido, service worker registrado, HTTPS, iconos en múltiples resoluciones. Debe ser instalable desde Chrome, Firefox y Safari en iOS y Android.
RNF07	Mantenibilidad	El código fuente deberá seguir una estructura modular con separación de responsabilidades. Se usarán nombres descriptivos y comentarios en funciones críticas. El repositorio incluirá README con instrucciones de instalación y variables de entorno requeridas.

7. Diseño del Sistema

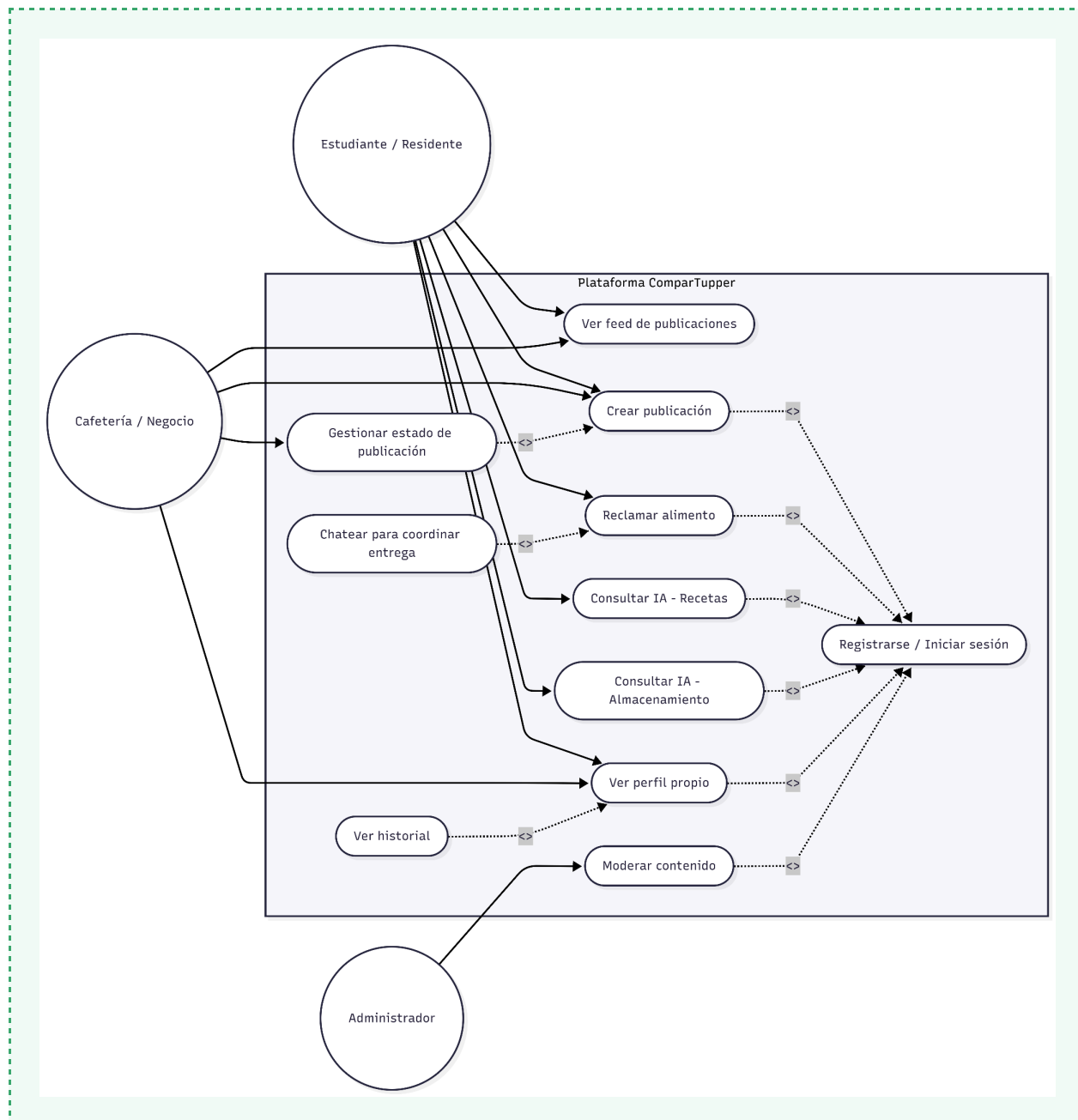
7.1 Arquitectura General

SmartFood sigue una arquitectura de aplicación web de tres capas desplegada en servicios cloud administrados. El frontend es una aplicación Next.js con capacidades PWA servida desde Vercel. El backend utiliza Supabase como Backend-as-a-Service (BaaS), que provee autenticación, base de datos PostgreSQL en tiempo real y almacenamiento de archivos. La IA se consume directamente desde el frontend a través de la API de Google Gemini Flash.



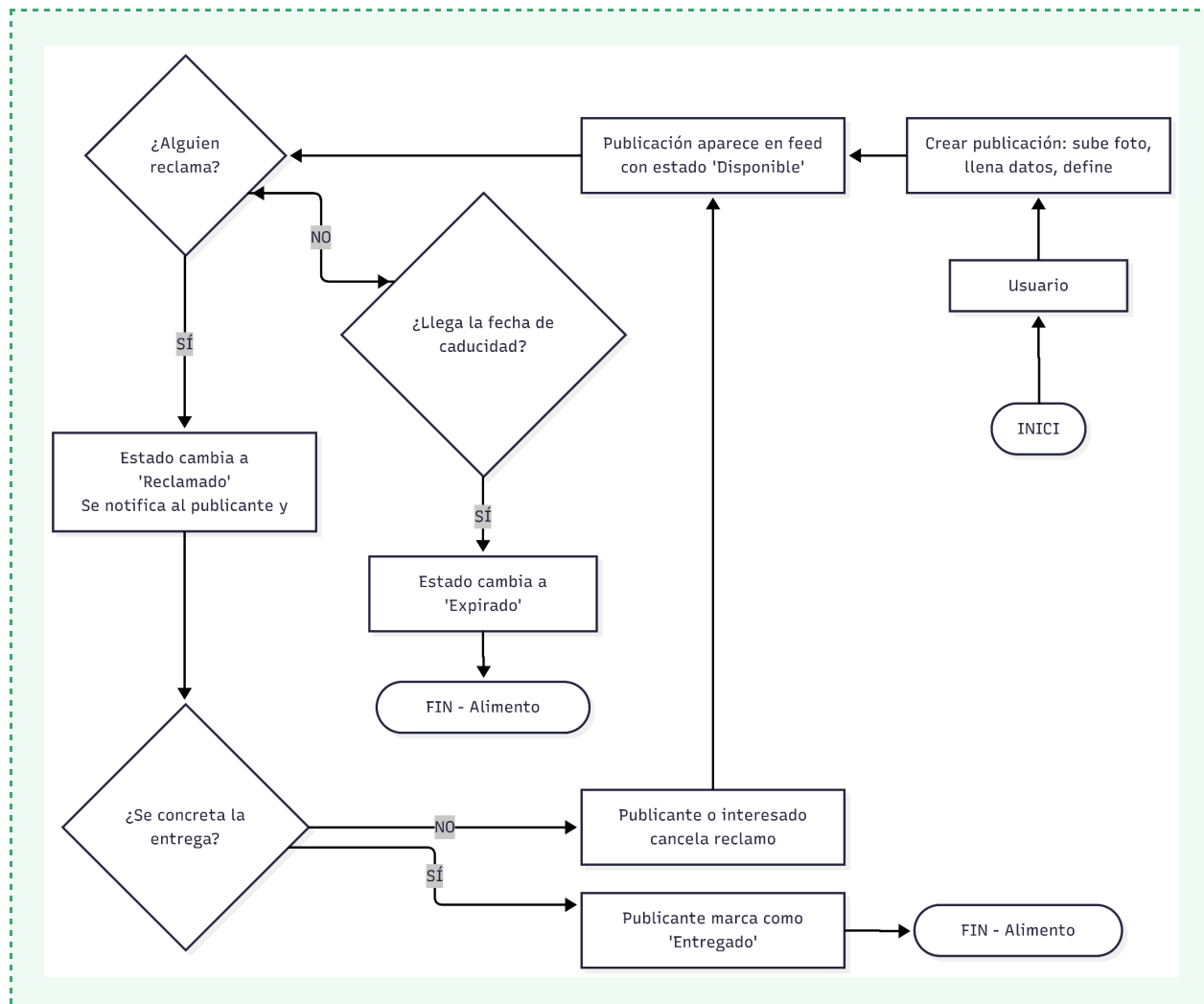
7.2 Diagrama de Casos de Uso

El siguiente diagrama muestra las interacciones entre los actores del sistema (Estudiante/Residente, Cafetería y Administrador) y las funcionalidades disponibles para cada uno:



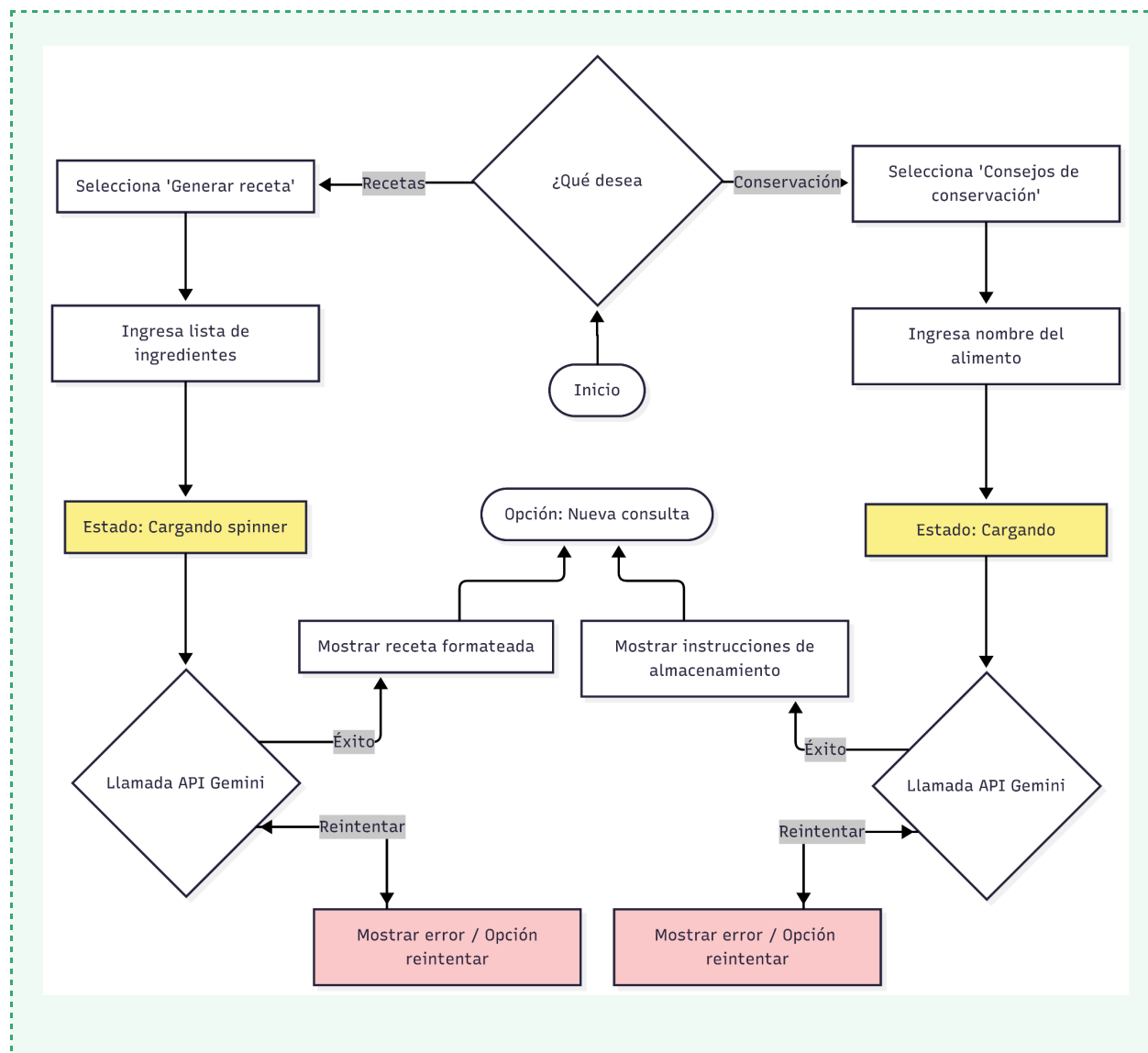
7.3 Diagrama de Flujo del Proceso Principal

El flujo central de la aplicación describe el ciclo de vida de una publicación de alimento, desde su creación hasta su resolución:



7.4 Diagrama de Flujo — Asistente IA

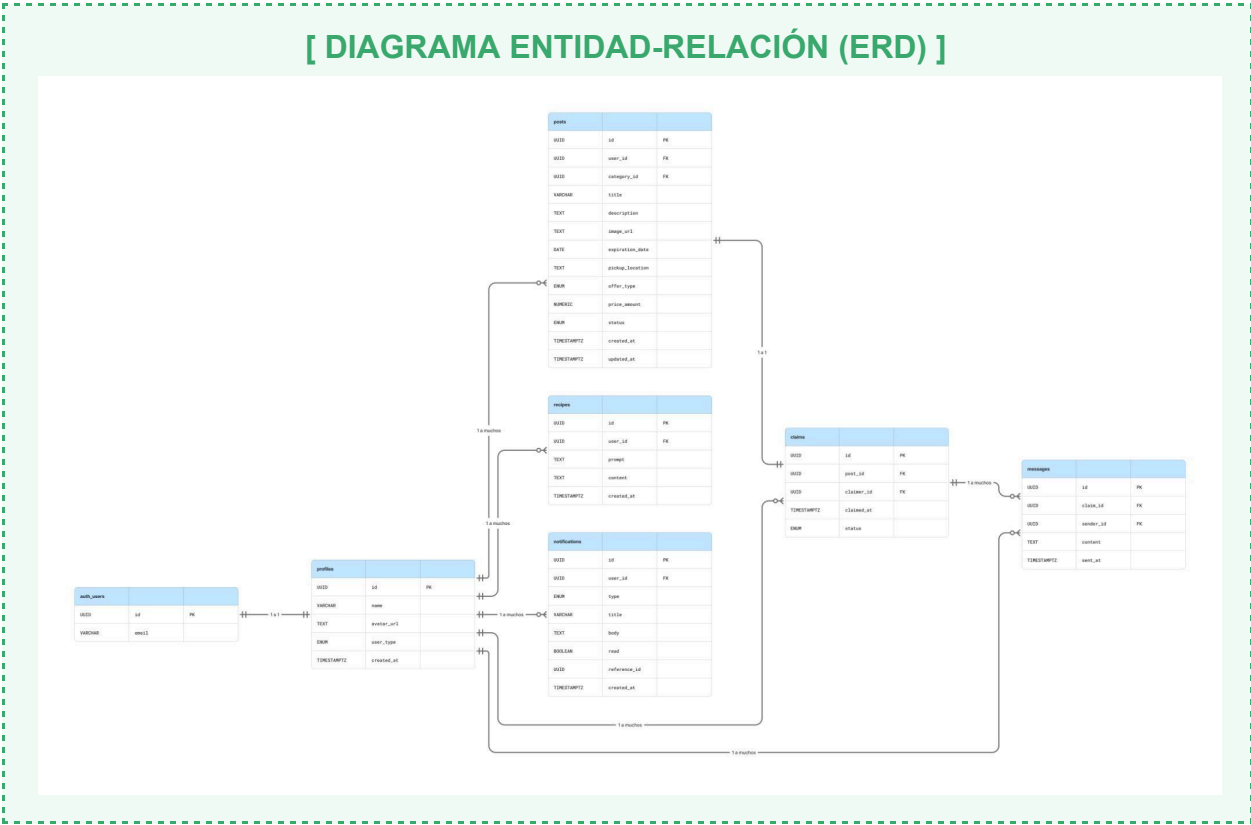
El flujo de interacción con el asistente de inteligencia artificial es compartido por los dos módulos (Recetas y Almacenamiento):



8. Modelo de Base de Datos

La base de datos utiliza PostgreSQL gestionado por Supabase. El modelo relacional contempla las siguientes entidades principales con sus atributos y relaciones:

8.1 Diagrama Entidad-Relación (ERD)



9. Diseño de Interfaz de Usuario

SmartFood adopta un enfoque mobile-first con una paleta visual verde y blanco que refuerza el concepto de sustentabilidad. La navegación principal se estructura en cuatro secciones accesibles desde una barra inferior fija (tab bar) optimizada para uso con el pulgar en el móvil.

9.1 Paleta de Colores y Tipografía

Elemento	Color / Valor	Uso
Primary	#2D7A4F (verde oscuro)	Botones principales, acentos, encabezados
Secondary	#3AA76D (verde medio)	Estados activos, badges, progreso
Background	#F9FAFB (gris muy claro)	Fondo general de la app
Surface	#FFFFFF (blanco)	Tarjetas, modales, formularios
Text Primary	#1E2A3A (azul muy oscuro)	Títulos y texto principal
Text Secondary	#6B7280 (gris medio)	Texto de apoyo, metadatos
Danger	#EF4444 (rojo)	Alertas de caducidad proxima
Tipografia	Inter (Google Fonts)	Fuente principal, pesos 400/500/700

9.2 Pantallas Principales — Mockups

Pantalla 1: Inicio de Sesión / Registro

SmartFood

Iniciar sesión

Registrarse

Email

Password

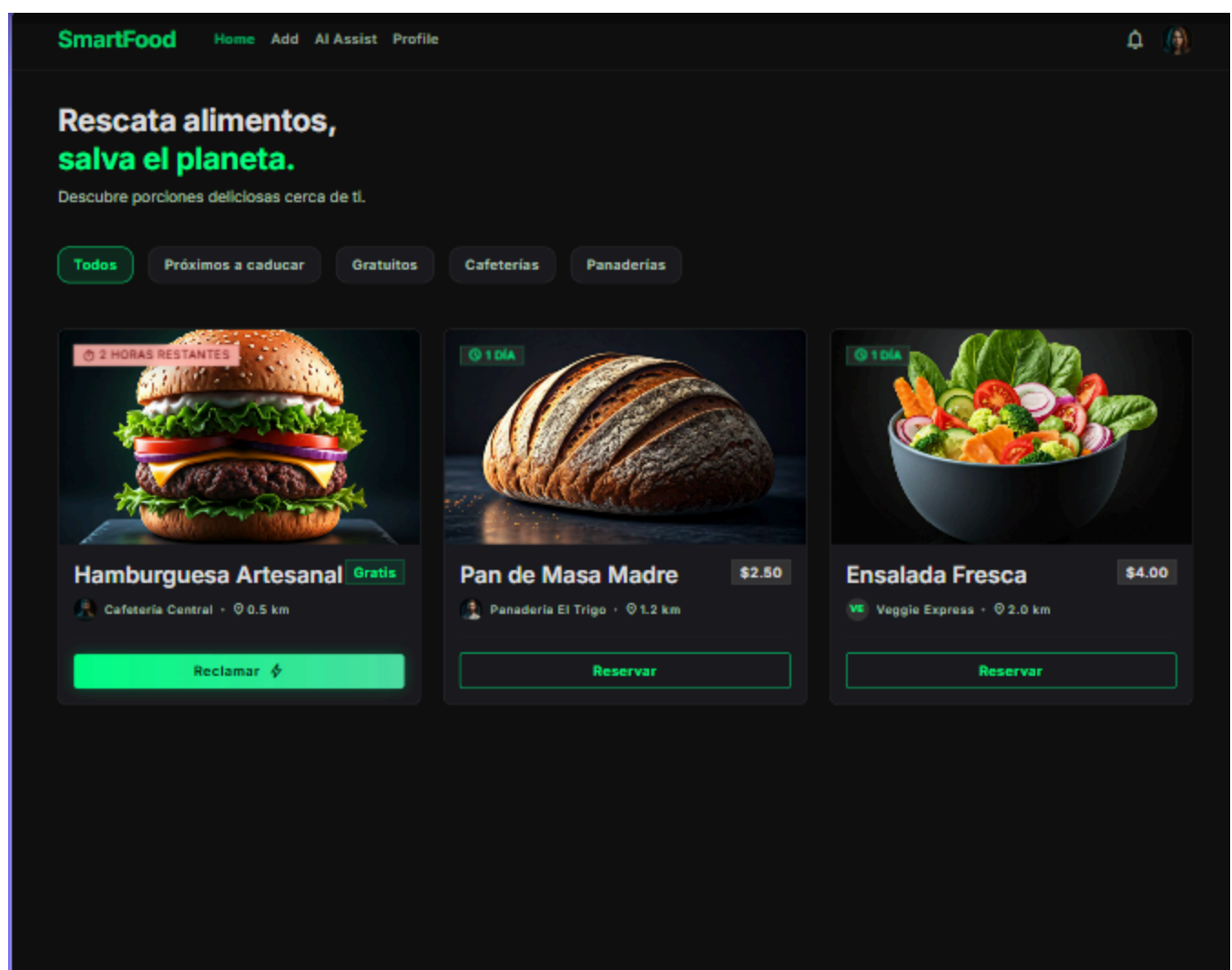
¿Olvidaste tu contraseña?

Entrar →

o

Continuar con Google

Pantalla 2: Feed Principal



Pantalla 3: Crear Publicación

Cancelar

Nueva publicación



Publicando como
Carlos Mendoza



Agregar foto

TÍTULO DEL ALIMENTO

Ej. Canasta de verduras orgánicas

DESCRIPCIÓN

Describe el estado, cantidad y por qué lo compartes...

FECHA DE VENCIMIENTO



05/19/2026



PUNTO DE ENTREGA



Ej. Estación Central o Calle Falsa 123

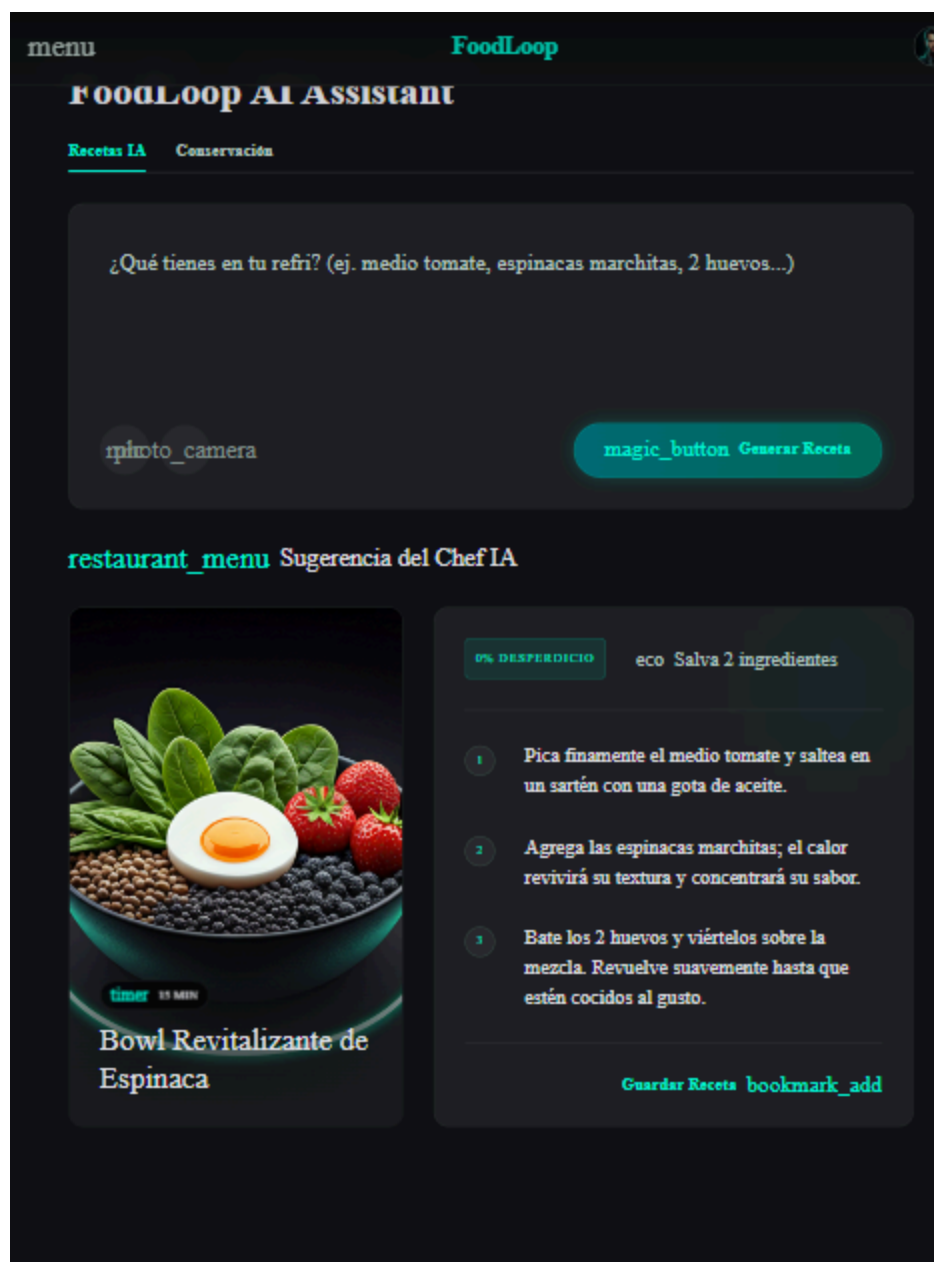
Tipo de oferta

Gratis

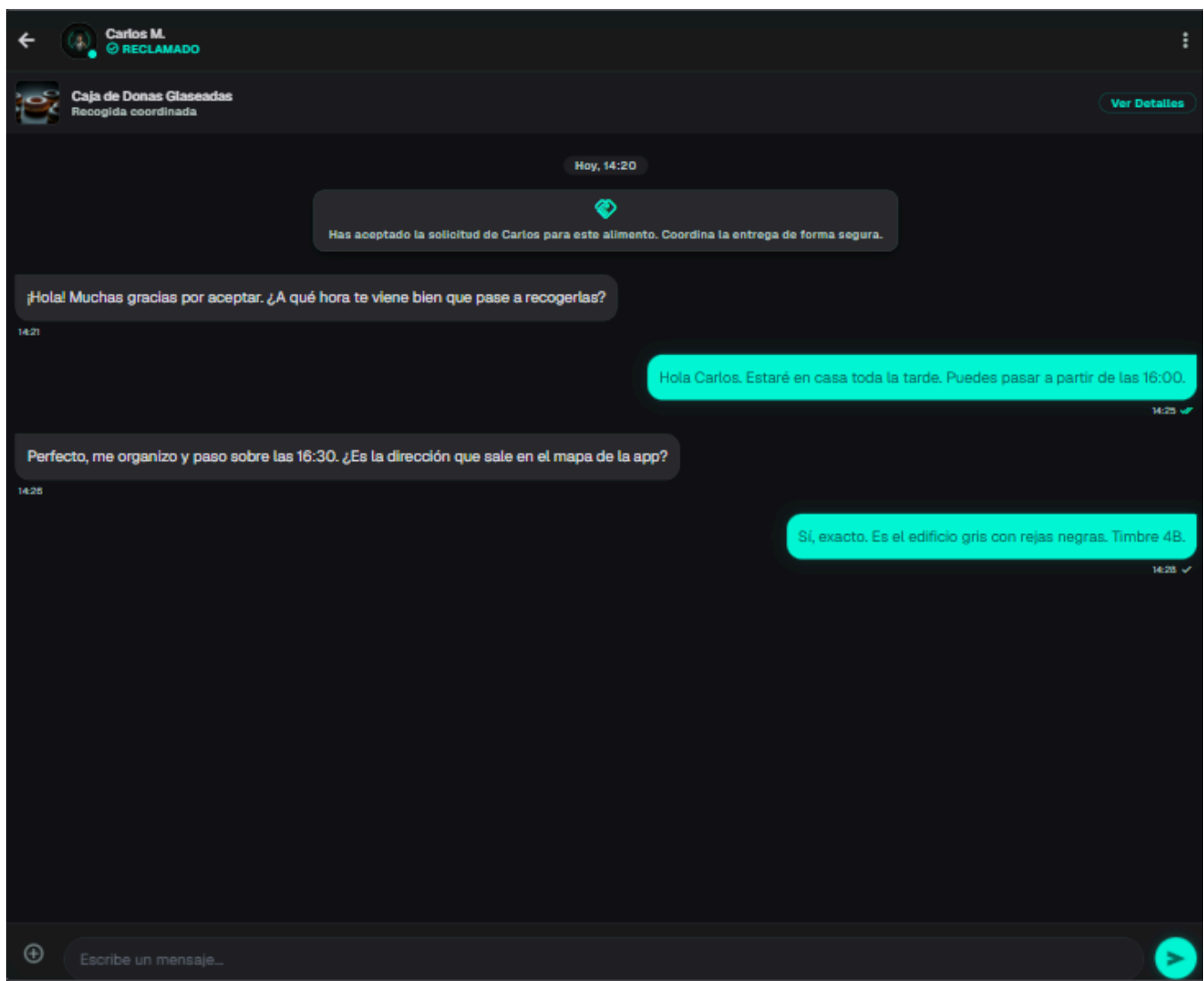
☐

Publicar

Pantalla 4: Asistente IA — Chef / Almacenamiento



Pantalla 5: Chat de Coordinacion



10. Stack Tecnológico

Capa	Tecnología	Justificación
Frontend Framework	Next.js 14 (App Router)	Permite SSR, SSG y rutas API en un solo framework. Soporte nativo para PWA y optimización de imágenes.
Lenguaje	TypeScript	Tipado estático reduce errores en tiempo de desarrollo. Mejor autocompletado y mantenibilidad.
Estilos	Tailwind CSS	Desarrollo rápido de UI responsive mobile-first. Consistencia visual sin CSS personalizado extenso.
Backend / BaaS	Supabase	PostgreSQL, autenticación, storage y suscripciones en tiempo real. Reduce el tiempo de desarrollo de backend.
Base de datos	PostgreSQL (via Supabase)	Base de datos relacional robusta con soporte para JSON, full-text search y Row Level Security (RLS).
Autenticación	Supabase Auth	OAuth con Google integrado. Manejo de sesiones JWT. Row Level Security nativa.
Storage	Supabase Storage	Almacenamiento de imágenes de publicaciones y fotos de perfil con CDN integrado.
Tiempo real	Supabase Realtime	Suscripciones WebSocket para notificaciones y chat en tiempo real sin configuración adicional.
Inteligencia Artificial	Google Gemini Flash API	Modelo rápido y eficiente para generación de texto. Latencia baja ideal para respuestas de recetas y consejos.
PWA	next-pwa / Service Worker	Librería que integra PWA en Next.js: manifest.json, service worker con caché offline, instalabilidad.
Despliegue	Vercel	Integración nativa con Next.js. Despliegue automático desde GitHub. CDN global, HTTPS automático.
Control de versiones	Git + GitHub	Repositorio público con historial de commits, ramas por funcionalidad y README.

11. Consideraciones de Implementación

11.1 Seguridad — Row Level Security (RLS)

Supabase permite definir políticas de acceso a nivel de fila en PostgreSQL. Se implementarán las siguientes políticas mínimas:

- **posts:** Cualquier usuario autenticado puede leer publicaciones con estado 'available'. Solo el creador puede actualizar o eliminar sus propias publicaciones.
- **claims:** El creador del reclamo y el dueño de la publicación pueden leer el reclamo. Solo el reclamante puede crear un reclamo.
- **messages:** Solo los dos participantes de un reclamo (publicante y reclamante) pueden leer y escribir mensajes en ese chat.
- **Notificaciones:** Cada usuario solo puede leer sus propias notificaciones.

11.2 Datos Simulados para el MVP

Durante el hackathon se utilizarán datos simulados (seed data) para poblar el feed con publicaciones de ejemplo. No se requieren sensores físicos ni datos reales. Se creará un script de seed para Supabase con al menos 15 publicaciones de ejemplo con diferentes estados, fechas de caducidad y tipos de usuario.

11.3 Rate Limiting para la API de IA

Para evitar el agotamiento de la cuota de la API de Gemini Flash, se implementará un control de uso a nivel de cliente que limite las solicitudes a un máximo de 10 por usuario por sesión. Se mostrará un mensaje informativo al usuario si alcanza el límite.

11.4 Manejo de Imágenes

Las imágenes de publicaciones se almacenarán en Supabase Storage en el bucket 'post-images'. Se aplicará una validación de tipo (solo JPG/PNG/WEBP) y tamaño máximo (5MB) antes de la subida. Next.js Image Optimization se usará para servir imágenes en formato WebP con los tamaños adecuados para cada dispositivo.